BVS1 – Betriebssysteme

Lecture 04 – I Have a Process: Integration and Diagnosis · Wintersemester 2026/2027

Stavros Kousidis

TH Köln · ICCT

Reconstruct the Process

No New Mechanism Today

Everything required for CP1 has already been introduced.

Today we ask:

> *Can we reconstruct, explain, and diagnose the complete process without being led through every step?*

Checkpoint 1:

> *I have a process.*

Rebuild the Story from Memory

Put these in a useful dependency order:

```
process lifecycle
saved context
separate stack
source → ELF
kernel regains control
entry function
context switch
exit status
```

Which concepts are **machine-level mechanisms**?

Which are **operating-system abstractions**?

The Smallest Complete Process

Without looking back, reconstruct the minimum information a process needs.

Possible questions:

- Where can it continue?
- Which stack belongs to it?
- What code should it run?
- Is it ready, running, or finished?
- How did it finish?

Which of these belong inside the saved context, and which do not?

Reconstruct the Lifecycle

Fill the missing labels:

```
unused
   ↓ ______
ready
   ↓ ______
running
   ↓ ______
ready
   ↓ ______
running
   ↓ ______
finished
```

Trace the Control Transfers

First Run

A fresh process has:

```
sp → top of process stack
ra → process_bootstrap
state = READY
```

Kernel switches to it.

Trace:

1. what is saved;
2. what is restored;
3. what `ret` does;
4. what code runs first.

Where did the first continuation come from?

Yield

Process body:

```
puts("A\n");
process_yield();
puts("B\n");
```

Observed:

```
A
<kernel output>
B
```

Explain why this observation is stronger evidence of resume than:

```
A
```

Exit

Suppose the process executes:

```
process_exit(42);
```

After the kernel regains control:

```
state       = FINISHED
exit_status = 42
```

Question:

The saved context may still contain **`ra`** *and* **`sp`***. Why must the kernel nevertheless refuse to resume it?*

Diagnose, Don’t Memorize

Failure A — Wrong Stack

Symptom:

```
process prints first message
process yields
kernel does some work
resume crashes or behaves strangely
```

Observation:

The restored `ra` looks plausible.

Question:

> *Which other saved value would you investigate first, and why?*

Failure B — Restart Instead of Resume

Expected:

```
process: first
kernel: back
process: second
```

Actual:

```
process: first
kernel: back
process: first
```

What conceptual mistake is more likely?

- wrong initial entry?

Failure C — A Dead Process Returns

Trace:

```
process exits 42
kernel observes FINISHED
kernel runs it again
```

What is wrong?

The machine state may still be restorable.

The **process abstraction** says it must not be.

Is It Really a Process?

Function Call or Process?

Case A:

```
hello();
```

Case B:

```
create hello
run
yield
kernel work
resume
exit
kernel observes status
```

Give two reasons why B represents independently managed execution and A does not.

Where Has Assembly Gone?

At the machine boundary:

```
save registers
restore registers
ret
```

Above it:

```
create process
run process
yield
resume
exit
```

> *We built an abstraction so that most reasoning can move upward into C and process concepts.*

CP1 Readiness

Behavioural Contract

A checker should be able to observe behaviour equivalent to:

```
process starts ........ OK
process yields ........ OK
kernel regains control  OK
process resumes ....... OK
exit status observed .. OK
```

The checker tests **behaviour**.

It does not prove understanding.

What You Must Be Able to Explain

At CP1, be ready to explain:

- what state survives a yield;
- why the process needs its own stack;
- first start vs. resume;
- yield vs. exit;
- why observed output shows real continuation;
- which small assembly boundary enables the C abstraction.

And be ready for one small equivalent variation or diagnosis.

Self-Check Before the Appointment

Can you do all four without notes?

1. draw kernel ↔ process control transfer;
2. trace `sp` and the continuation through yield/resume;
3. predict output after moving one message across the yield;
4. diagnose one broken lifecycle/state transition.

If not:

Use lab support to work on the checkpoint goal.

Bridge

What Problem Appears Next?

With one process, the kernel can choose explicitly when to run it again.

Now imagine:

```
process A never calls yield
process B is ready
```

What happens to B?

How can the OS regain control when the running process does not cooperate?

The Abstraction Climb So Far

```
source → RV64 program
        ↑
saved execution context
        ↑
process lifecycle
        ↑
CP1 — I have a process.
```

You should still understand the lower layers.

But you should no longer need to reason in assembly every time you say:

```
resume the process
```